WOXSEN UNIVERSITY, HYDERABAD

DATABASE MANAGEMENT SYSTEMS (PBL)

Project Report

PROJECT 22

Campus Placement and Recruitment Drive Management System

Oracle 19c | Java JDBC | JSP Web Portal

Submitted by

C. Sai Ridhi

Roll No: 25WU0102055

B.Tech, Woxsen University

Submitted to

Prof. Sahiti

October 2026

Abstract

Campus placement involves many parties and many rules: companies announce job profiles, students apply, eligibility criteria must be checked, several selection rounds have to be cleared in order, and offers must be tracked. When this is handled with spreadsheets, data is duplicated, rules are applied inconsistently and statistics are hard to obtain.

This project presents a Campus Placement and Recruitment Drive Management System built on Oracle Database 19c. The schema has thirteen tables in Third Normal Form covering departments, students, academic records, skills, companies, job profiles, drives, eligibility criteria, applications, selection rounds, results and offers. Declarative constraints protect basic integrity, while two PL/SQL triggers enforce the business rules that need information from other tables: a student can apply only to a drive whose CGPA and backlog criteria are met (ORA-20001), and a student can enter a round only after passing the previous round (ORA-20002 and ORA-20003).

The database is complemented by analytical queries, two reporting views, a Java JDBC console application and a JSP web portal with student and administrator dashboards. The student dashboard displays every drive with its eligibility criteria and tells each student whether they qualify; the administrator dashboard records round results and issues offers. Testing showed that an ineligible student is rejected by the database itself, whichever application is used.

Keywords: placement management, Oracle 19c, normalization, triggers, JDBC, JSP.

Contents

1. Introduction....4
1.1 Background....4
1.2 Problem Statement....4
1.3 Motivation....4
1.4 Organization of the Report....4
2. Objectives, Scope and Technology....4
2.1 Objectives....4
2.2 Scope....5
2.3 Technology Stack....5
3. System Requirements....5
3.1 Stakeholders....5
3.2 Functional Requirements....5
3.3 Business Rules....6
3.4 Non-Functional Requirements....6
4. Database Design....6
4.1 Conceptual Design: Entity-Relationship Model....6
4.2 Entities and Relationships....7
4.3 Relational Schema....7
4.4 Normalization....7
5. Implementation in Oracle 19c....8
5.1 Environment Setup....8
5.2 Order of Table Creation....8
5.3 Data Dictionary....8
5.4 Integrity Constraints....10
5.5 Trigger 1: Eligibility Check....10
5.6 Trigger 2: Round Sequence Check....11
5.7 The Reserved-Name Problem....12
6. Sample Data and Testing....13
6.1 Sample Data....13
6.2 Test Cases....13
6.3 Dry Run of the Eligibility Trigger....14
7. Queries and Views....14
7.1 Analytical Queries....14
7.2 Views....15
8. Application Development....15
8.1 Architecture....15
8.2 Java Console Application....16
8.3 JSP Web Portal....16
8.4 Deployment Steps....16
8.5 Security Notes....17
9. Challenges and Resolutions....17
10. Conclusion and Future Work....17
10.1 Conclusion....17
10.2 Future Work....18
References....18
Appendix A: Complete DDL Script....18
Appendix B: Sample Data....20

1. Introduction

1.1 Background

Every year, the placement cell of an engineering institution coordinates a large number of interactions between students and recruiting companies. A company announces a job profile, fixes a campus drive date, publishes eligibility rules, and then conducts several selection rounds such as an aptitude test, a technical interview and an HR interview. Students apply, appear for the rounds, and the successful ones receive offers. In many colleges this entire process is still managed with spreadsheets, e-mail threads and notice boards.

Such manual handling creates predictable problems. The same student data is copied between several files and drifts out of date. Eligibility rules are checked by eye, so an ineligible student may be shortlisted by mistake. Round-wise results are scattered across documents, which makes it hard to answer simple questions such as "how many students of the Electronics department have received offers?". A relational database with enforced rules is the natural remedy, and this project builds one.

1.2 Problem Statement

To design and implement a **Campus Placement and Recruitment Drive Management System** (Project 22 of the DBMS Project Based Learning course) that stores student, company, drive and selection-process data in a normalized Oracle database, automatically enforces eligibility and round-sequence rules, produces placement statistics, and exposes the data through a student dashboard and an administrator (placement cell) dashboard.

1.3 Motivation

The project was chosen because it exercises almost every topic of a first course in database systems within one realistic scenario: entity-relationship modelling, mapping to relations, normalization, integrity constraints, PL/SQL triggers, joins, sub-queries, aggregation, views, transactions and connecting a database to an application through JDBC. The domain is also familiar to every student on campus, which makes the requirements easy to validate.

1.4 Organization of the Report

Section 2 states the objectives, scope and technology stack. Section 3 lists the system requirements. Section 4 presents the conceptual and logical database design, and Section 5 describes the Oracle implementation including the data dictionary and triggers. Section 6 documents sample data and testing, Section 7 the queries and views, and Section 8 the console and web applications. Section 9 records the problems met during development and how they were solved, and Section 10 concludes with future enhancements. Appendices A and B reproduce the full DDL script and the sample data.

2. Objectives, Scope and Technology

2.1 Objectives

- Model the placement process as an entity-relationship diagram and map it to a relational schema in Third Normal Form (3NF).
- Implement the schema in Oracle Database 19c with primary keys, foreign keys, unique, check and default constraints.
- Enforce business rules inside the database through triggers, so that no application can bypass them.

- Provide meaningful analytical queries and reporting views for the placement cell.
- Build a Java JDBC console application and a JSP web portal with separate student and administrator dashboards that display eligibility criteria for every drive.

2.2 Scope

The system covers the complete lifecycle from a student applying to a drive to an offer being issued. It manages departments, students and their academic records, skills, companies, job profiles, drives, eligibility criteria, applications, selection rounds, round results and offers. Payment, resume file storage, e-mail notification and multi-campus operation are outside the scope of this version.

2.3 Technology Stack

Table 2.1: Technology stack

Component	Choice	Purpose
Database	Oracle Database 19c Enterprise Edition (19.3.0.0.0), pluggable database ORCLPDB	Storage, constraints, triggers, views
Client tool	SQL*Plus	Running DDL, DML and queries
Console application	Java with JDBC (ojdbc8)	Command-line access for the placement cell
Web portal	JSP and JDBC on Apache Tomcat (XAMPP)	Student and admin dashboards
Styling	Hand-written CSS	Dashboard layout and badges
Diagrams	ER notation, flowcharts	Design documentation

3. System Requirements

3.1 Stakeholders

Two kinds of users interact with the system. **Students** view the list of upcoming drives together with the eligibility criteria, see whether they themselves qualify, apply to eligible drives and track their results. The **placement officer** (administrator) reviews the applicants of each drive, records the result of every round, issues offers and consults the placement statistics.

3.2 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR-1	The system shall store department, student, academic record and skill information.
FR-2	The system shall store companies, their job profiles and the drives conducted for those profiles.
FR-3	Each drive shall have eligibility criteria (minimum CGPA and maximum number of active backlogs) visible to students.
FR-4	A student shall be able to apply to a drive only if the criteria are met, and only once per drive.
FR-5	Each drive shall have an ordered list of rounds; a student may enter a round only after passing the previous one.
FR-6	The administrator shall record round results and issue at most one offer per application.
FR-7	The system shall report placement statistics by department and by company.

3.3 Business Rules

- A student has exactly one academic record, with a CGPA between 0 and 10 and a non-negative backlog count.
- A student is eligible for a drive if CGPA >= min_cgpa and backlogs <= max_backlogs.
- A student can apply to a given drive only once.
- Round numbers are unique within a drive; the result of a round is either Pass or Fail.
- An application can have at most one offer, whose status is Pending, Accepted or Rejected.

3.4 Non-Functional Requirements

- **Integrity:** rules are enforced in the database layer, independent of the front end.
- **Consistency:** multi-step operations are executed as transactions with commit and rollback.
- **Usability:** dashboards show eligibility and status with clear colour-coded badges.
- **Maintainability:** a layered application structure separates presentation, service logic and data access.
- **Portability:** all components run on a single laptop (Oracle 19c, Java, Tomcat) without extra infrastructure.

4. Database Design

4.1 Conceptual Design: Entity-Relationship Model

The ER model contains thirteen entity sets. The central entity is **Application**, which links a Student to a Drive and from which the selection results and the offer hang. Figure 4.1 shows the entities and the cardinality of each relationship; the Application entity is highlighted.

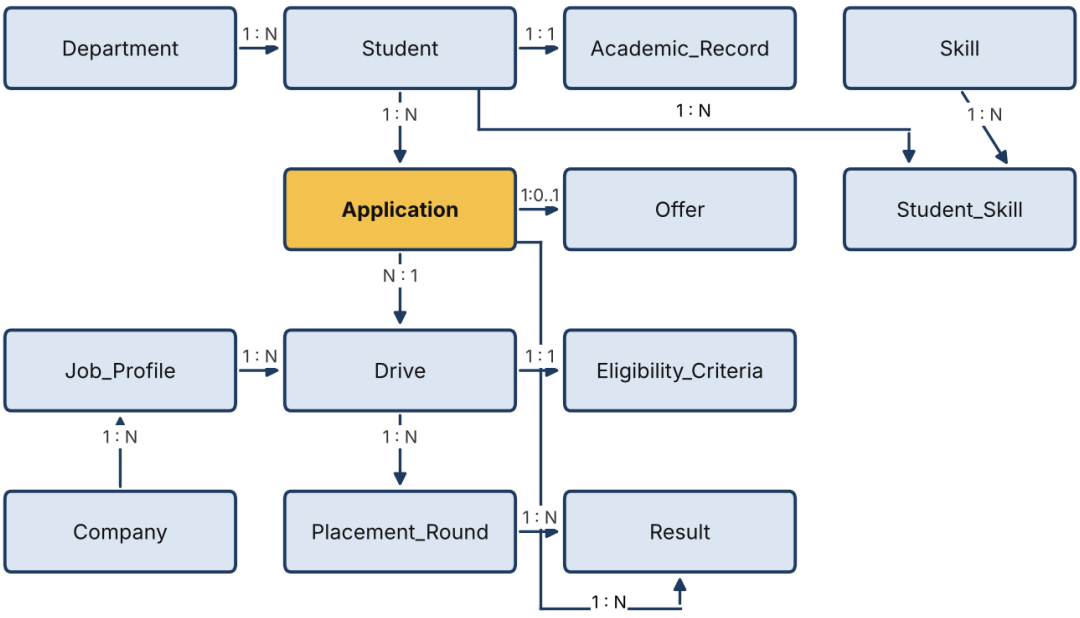


Figure 4.1: Entity-relationship overview of the placement system

4.2 Entities and Relationships

Table 4.1: Relationships and cardinalities

Relationship	Cardinality	Explanation
Department - Student	1 : N	A department has many students; each student belongs to one department.
Student - Academic_Record	1 : 1	Each student has one record of CGPA and backlogs.
Student - Skill	M : N	Resolved through the Student_Skill junction table.
Company - Job_Profile	1 : N	A company offers several roles.
Job_Profile - Drive	1 : N	A role can be recruited for in more than one drive.
Drive - Eligibility_Criteria	1 : 1	Every drive has one set of criteria.
Student - Application - Drive	M : N	Application is the associative entity with the applied-on date.
Drive - Placement_Round	1 : N	A drive has an ordered list of rounds.
Application - Result - Placement_Round	M : N	A result records the outcome of one application in one round.
Application - Offer	1 : 0..1	An application yields at most one offer.

4.3 Relational Schema

Mapping the ER model to relations gives the following schema. Primary keys are underlined in the usual notation and shown here in capitals; foreign keys are marked FK.

```
Department(DEPT_ID, dept_name, dept_code)
Student(STUDENT_ID, name, dept_id FK, email)
Academic_Record(RECORD_ID, student_id FK, cgpa, backlogs)
Skill(SKILL_ID, skill_name)
Student_Skill(STUDENT_ID FK, SKILL_ID FK)
Company(COMPANY_ID, name, sector)
Job_Profile(PROFILE_ID, company_id FK, role, package_lpa)
Drive(DRIVE_ID, profile_id FK, drive_date)
Eligibility_Criteria(DRIVE_ID FK, min_cgpa, max_backlogs)
Application(APP_ID, student_id FK, drive_id FK, applied_on)
Placement_Round(ROUND_ID, drive_id FK, round_no, round_type)
Result(APP_ID FK, ROUND_ID FK, status)
Offer(OFFER_ID, app_id FK, offer_date, status)
```

Listing 4.1: Relational schema

4.4 Normalization

All relations satisfy First Normal Form because every attribute is atomic: a student's skills are not stored as a comma-separated list but in the Student_Skill table. They satisfy Second Normal Form because in every relation with a composite key (Student_Skill and Result) there are no non-key attributes that depend on only part of the key; Result has a single non-key attribute, status, which depends on the whole pair (app_id, round_id).

Third Normal Form is achieved by moving every transitive dependency into its own relation. For example, a student's department name depends on dept_id and not directly on student_id, so it lives in Department. The package of a role depends on the job profile rather than on the drive, so it lives in Job_Profile; the company sector depends on the company, so it lives in Company. The functional dependencies that drive the design are listed below.

Table 4.2: Functional dependencies

Relation	Functional dependencies
Student	student_id -> name, dept_id, email

Relation	Functional dependencies
Academic_Record	record_id -> student_id, cgpa, backlogs; student_id -> record_id
Job_Profile	profile_id -> company_id, role, package_lpa
Eligibility_Criteria	drive_id -> min_cgpa, max_backlogs
Application	app_id -> student_id, drive_id, applied_on; (student_id, drive_id) -> app_id
Result	(app_id, round_id) -> status
Offer	offer_id -> app_id, offer_date, status; app_id -> offer_id

In every dependency the determinant is a candidate key, so each relation is in Boyce-Codd Normal Form as well. The only deliberate redundancy is that Placement_Round stores drive_id, which is needed to scope round numbers to a drive.

5. Implementation in Oracle 19c

5.1 Environment Setup

The database runs as Oracle Database 19c Enterprise Edition (19.3.0.0.0) with a pluggable database named ORCLPDB. Work is done as the SYSTEM user connected to ORCLPDB with SQL*Plus. Two practical points mattered during setup. First, the pluggable database must be open before connecting (ALTER PLUGGABLE DATABASE ORCLPDB OPEN). Second, tables created as SYSTEM live in the SYSTEM schema, so a session connected as SYS must prefix table names with SYSTEM or, preferably, reconnect as SYSTEM.

5.2 Order of Table Creation

Foreign keys can only point to tables that already exist, so the DDL script creates parent tables first: Department, Student, Academic_Record, Skill, Student_Skill, Company, Job_Profile, Drive, Eligibility_Criteria, Application, Placement_Round, Result and finally Offer. The complete script is reproduced in Appendix A.

5.3 Data Dictionary

Table 5.1: Department

Column	Data type	Constraints
dept_id	NUMBER	PRIMARY KEY
dept_name	VARCHAR2(50)	NOT NULL
dept_code	VARCHAR2(10)	UNIQUE

Table 5.2: Student

Column	Data type	Constraints
student_id	NUMBER	PRIMARY KEY
name	VARCHAR2(60)	NOT NULL
dept_id	NUMBER	NOT NULL, FK to Department
email	VARCHAR2(80)	UNIQUE

Table 5.3: Academic_Record

Column	Data type	Constraints
record_id	NUMBER	PRIMARY KEY
student_id	NUMBER	NOT NULL, UNIQUE, FK to Student

Column	Data type	Constraints
cgpa	NUMBER(3,2)	NOT NULL, CHECK between 0 and 10
backlogs	NUMBER(2)	DEFAULT 0, NOT NULL, CHECK >= 0

Table 5.4: Skill

Column	Data type	Constraints
skill_id	NUMBER	PRIMARY KEY
skill_name	VARCHAR2(50)	NOT NULL, UNIQUE

Table 5.5: Student_Skill

Column	Data type	Constraints
student_id	NUMBER	Part of composite PRIMARY KEY, FK to Student
skill_id	NUMBER	Part of composite PRIMARY KEY, FK to Skill

Table 5.6: Company

Column	Data type	Constraints
company_id	NUMBER	PRIMARY KEY
name	VARCHAR2(80)	NOT NULL
sector	VARCHAR2(50)	-

Table 5.7: Job_Profile

Column	Data type	Constraints
profile_id	NUMBER	PRIMARY KEY
company_id	NUMBER	NOT NULL, FK to Company
role	VARCHAR2(60)	NOT NULL
package_lpa	NUMBER(5,2)	-

Table 5.8: Drive

Column	Data type	Constraints
drive_id	NUMBER	PRIMARY KEY
profile_id	NUMBER	NOT NULL, FK to Job_Profile
drive_date	DATE	NOT NULL

Table 5.9: Eligibility_Criteria

Column	Data type	Constraints
drive_id	NUMBER	PRIMARY KEY, FK to Drive (1:1)
min_cgpa	NUMBER(3,2)	NOT NULL, CHECK between 0 and 10
max_backlogs	NUMBER(2)	DEFAULT 0

Table 5.10: Application

Column	Data type	Constraints
app_id	NUMBER	PRIMARY KEY
student_id	NUMBER	NOT NULL, FK to Student
drive_id	NUMBER	NOT NULL, FK to Drive

Column	Data type	Constraints
applied_on	DATE	DEFAULT SYSDATE
(student_id, drive_id)	-	UNIQUE: one application per drive

Table 5.11: Placement_Round

Column	Data type	Constraints
round_id	NUMBER	PRIMARY KEY
drive_id	NUMBER	NOT NULL, FK to Drive
round_no	NUMBER(2)	NOT NULL
round_type	VARCHAR2(30)	NOT NULL
(drive_id, round_no)	-	UNIQUE: round numbers per drive

Table 5.12: Result

Column	Data type	Constraints
app_id	NUMBER	Part of composite PRIMARY KEY, FK to Application
round_id	NUMBER	Part of composite PRIMARY KEY, FK to Placement_Round
status	VARCHAR2(10)	NOT NULL, CHECK IN (Pass, Fail)

Table 5.13: Offer

Column	Data type	Constraints
offer_id	NUMBER	PRIMARY KEY
app_id	NUMBER	NOT NULL, UNIQUE, FK to Application
offer_date	DATE	DEFAULT SYSDATE
status	VARCHAR2(10)	DEFAULT Pending, CHECK IN (Pending, Accepted, Rejected)

5.4 Integrity Constraints

Declarative constraints are preferred wherever they can express a rule because the optimizer and the database enforce them uniformly. Primary keys guarantee row identity, foreign keys guarantee that no application refers to a missing student or drive, UNIQUE constraints implement the one-application-per-drive and one-offer-per-application rules, and CHECK constraints restrict CGPA to 0-10, round status to Pass or Fail, and offer status to Pending, Accepted or Rejected. Rules that need to look at other tables, namely eligibility and round order, cannot be written as CHECK constraints in Oracle and are therefore implemented as triggers.

5.5 Trigger 1: Eligibility Check

The trigger trg_check_eligibility fires BEFORE INSERT on Application. For the incoming row it reads the student's CGPA and backlog count from Academic_Record and the drive's limits from Eligibility_Criteria. If the CGPA is below the minimum or the backlogs exceed the maximum, it raises application error ORA-20001 and the insert is rejected. The logic is shown in Figure 5.1 and the code in Listing 5.1.

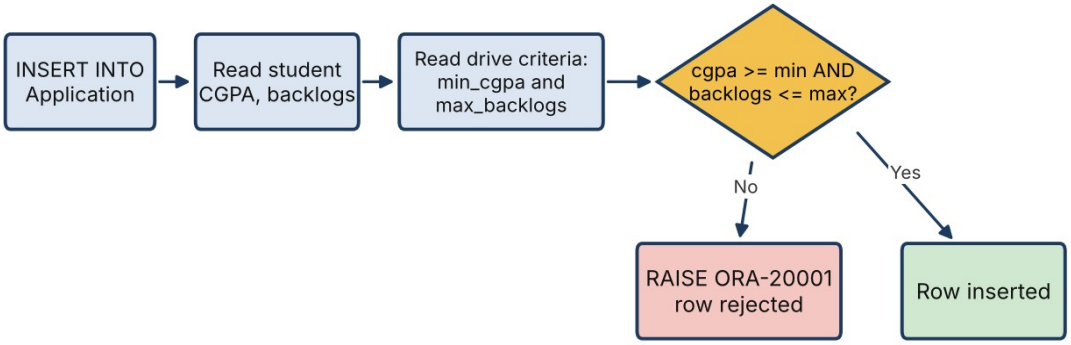


Figure 5.1: Logic of trg_check_eligibility

```
CREATE OR REPLACE TRIGGER trg_check_eligibility
BEFORE INSERT ON Application
FOR EACH ROW
DECLARE
    v_cgpa      Academic_Record.cgpa%TYPE;
    v_backlogs  Academic_Record.backlogs%TYPE;
    v_min_cgpa  Eligibility_Criteria.min_cgpa%TYPE;
    v_max_back  Eligibility_Criteria.max_backlogs%TYPE;
BEGIN
    SELECT cgpa, backlogs INTO v_cgpa, v_backlogs
    FROM Academic_Record WHERE student_id = :NEW.student_id;

    SELECT min_cgpa, max_backlogs INTO v_min_cgpa, v_max_back
    FROM Eligibility_Criteria WHERE drive_id = :NEW.drive_id;

    IF v_cgpa < v_min_cgpa OR v_backlogs > v_max_back THEN
        RAISE_APPLICATION_ERROR(-20001, 'Student does not meet eligibility criteria for this
drive.');
    END IF;
END;
/
```

Listing 5.1: trg_check_eligibility

5.6 Trigger 2: Round Sequence Check

The trigger trg_check_round_sequence fires BEFORE INSERT on Result. It looks up the round number and drive of the round being entered. For round 1 nothing more is checked. For later rounds it finds the previous round of the same drive and the student's result in it. If that result is not Pass the trigger raises ORA-20002; if no previous result exists at all (the NO_DATA_FOUND exception) it raises ORA-20003. This guarantees that students progress through the rounds strictly in order. See Figure 5.2.

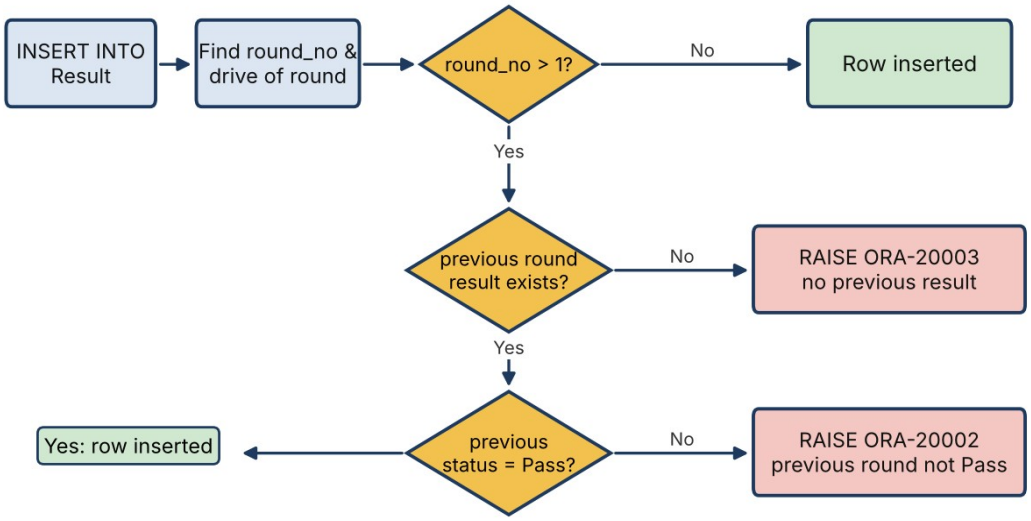


Figure 5.2: Logic of trg_check_round_sequence

```
CREATE OR REPLACE TRIGGER trg_check_round_sequence
BEFORE INSERT ON Result
FOR EACH ROW
DECLARE
    v_round_no      Placement_Round.round_no%TYPE;
    v_drive_id      Drive.drive_id%TYPE;
    v_prev_round_id Placement_Round.round_id%TYPE;
    v_prev_status   Result.status%TYPE;
BEGIN
    SELECT round_no, drive_id INTO v_round_no, v_drive_id
    FROM Placement_Round WHERE round_id = :NEW.round_id;

    IF v_round_no > 1 THEN
        SELECT round_id INTO v_prev_round_id
        FROM Placement_Round WHERE drive_id = v_drive_id AND round_no = v_round_no - 1;

        SELECT status INTO v_prev_status
        FROM Result WHERE app_id = :NEW.app_id AND round_id = v_prev_round_id;

        IF v_prev_status != 'Pass' THEN
            RAISE_APPLICATION_ERROR(-20002, 'Cannot enter this round without passing the
previous round.');
        END IF;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20003, 'Previous round result not found - student has not
cleared the prior stage.');
END;
/
```

Listing 5.2: trg_check_round_sequence

5.7 The Reserved-Name Problem

The table for selection rounds was first named Round. Creating the second trigger then failed with "created with compilation errors". Running SHOW ERRORS revealed PLS-00225: ROUND is the name of a built-in Oracle SQL function, so inside PL/SQL the identifier Round.round_no was resolved as a function call rather than a table reference. The table was renamed Placement_Round and the trigger recreated, after which it compiled cleanly. The lesson is to avoid reserved or built-in names for schema objects.

6. Sample Data and Testing

6.1 Sample Data

To exercise the schema, a small but representative data set was loaded: three departments, five students with academic records, four skills, three companies with one job profile each, three drives with eligibility criteria and rounds, four applications, nine round results and three offers. The complete INSERT statements are given in Appendix B.

Table 6.1: Students and academic records

ID	Name	Department	CGPA	Backlogs
101	Aditi Rao	Computer Science	8.75	0
102	Rahul Verma	Computer Science	7.20	1
103	Sneha Iyer	Electronics	9.10	0
104	Karthik Nair	Electronics	6.80	2
105	Priya Menon	Mechanical	8.05	0

Table 6.2: Drives and eligibility criteria

Drive	Company	Role	Package (LPA)	Date	Min CGPA	Max backlogs
1	TechNova Solutions	Software Engineer	8.50	15-OCT-2026	7.00	1
2	GreenGrid Energy	Electrical Design Engineer	6.75	20-OCT-2026	6.50	2
3	FinEdge Analytics	Data Analyst	7.20	25-OCT-2026	8.00	0

The data is designed so that eligibility matters. Drive 3 requires a CGPA of at least 8.00 with no backlogs, so only students 101, 103 and 105 qualify; Karthik Nair (6.80, 2 backlogs) does not. Drive 1 accepts Rahul Verma even with one backlog because its limit is 1.

Table 6.3: Rounds per drive

Drive	Round 1	Round 2	Round 3
1	Aptitude	Technical	HR
2	Aptitude	Technical	-
3	Technical	HR	-

6.2 Test Cases

Table 6.4: Test cases

ID	Test	Expected	Outcome
TC-1	Student 104 (CGPA 6.80, 2 backlogs) applies to drive 3 (min 8.00, 0 backlogs)	Rejected with ORA-20001	Observed in SQL*Plus and again through the Java console application
TC-2	Record a round-3 result for application 2 when the prior stage was not cleared	Rejected by round-sequence trigger	Observed: ORA-20003 "Previous round result not found - student has not cleared the prior stage." (ORA-06512 at TRG_CHECK_ROUND_SEQUENCE, line 23)

ID	Test	Expected	Outcome
TC-3	Eligible student 101 applies to drive 3	Application accepted	Eligible list for drive 3 showed students 101, 103 and 105
TC-4	A second application by the same student to the same drive	Rejected with ORA-00001	Enforced by UNIQUE (student_id, drive_id); the portal shows "already applied"
TC-5	Insert CGPA 11.0 into Academic_Record	Rejected by CHECK constraint	Enforced by the declared constraint (ORA-02290)
TC-6	Issue a second offer for the same application	Rejected with ORA-00001	Enforced by UNIQUE on Offer.app_id; the portal shows "already has an offer"

Test cases TC-1 and TC-2 were executed and observed during development. TC-4 to TC-6 are guaranteed by declared constraints and are handled in the application layer by mapping the Oracle error code to a friendly message.

6.3 Dry Run of the Eligibility Trigger

For TC-1 the trigger reads cgpa = 6.80 and backlogs = 2 for student 104, then min_cgpa = 8.00 and max_backlogs = 0 for drive 3. The condition 6.80 < 8.00 is true, so the OR expression is true and RAISE_APPLICATION_ERROR(-20001, ...) aborts the INSERT; no row is added to Application. For student 105 (8.05, 0 backlogs) both comparisons are false, the trigger exits quietly and the row is inserted.

7. Queries and Views

7.1 Analytical Queries

Q1. Students with the company and role they applied for (six-table join).

```
SELECT s.name, d.dept_name, c.name AS company, jp.role, dr.drive_date
FROM Application a
JOIN Student s       ON a.student_id = s.student_id
JOIN Department d    ON s.dept_id    = d.dept_id
JOIN Drive dr        ON a.drive_id   = dr.drive_id
JOIN Job_Profile jp  ON dr.profile_id = jp.profile_id
JOIN Company c       ON jp.company_id = c.company_id;
```

Listing 7.1: Query Q1

The query returned four rows: Aditi Rao and Rahul Verma for TechNova Software Engineer (15-OCT-26), Sneha Iyer of Electronics for GreenGrid Electrical Design Engineer (20-OCT-26), and Priya Menon of Mechanical for FinEdge Data Analyst (25-OCT-26).

Q2. Department-wise number of offers (aggregation with GROUP BY).

```
SELECT d.dept_name, COUNT(o.offer_id) AS offers
FROM Offer o
JOIN Application a ON o.app_id = a.app_id
JOIN Student s     ON a.student_id = s.student_id
JOIN Department d  ON s.dept_id = d.dept_id
GROUP BY d.dept_name;
```

Listing 7.2: Query Q2

Result: Electronics 1, Computer Science 1 and Mechanical 1.

Q3. Students who cleared the aptitude round of drive 1 (join with a filter).

```
SELECT s.name
```

```
FROM Result r
JOIN Application a      ON r.app_id = a.app_id
JOIN Student s          ON a.student_id = s.student_id
JOIN Placement_Round pr ON r.round_id = pr.round_id
WHERE pr.drive_id = 1 AND pr.round_no = 1 AND r.status = 'Pass';
```

Listing 7.3: Query Q3

Result: Aditi Rao and Rahul Verma.

Q4. Students whose CGPA is above the average (sub-query).

```
SELECT s.name, ar.cgpa
FROM Student s JOIN Academic_Record ar ON s.student_id = ar.student_id
WHERE ar.cgpa > (SELECT AVG(cgpa) FROM Academic_Record)
ORDER BY ar.cgpa DESC;
```

Listing 7.4: Query Q4

The average CGPA is 7.98, so the result is Sneha Iyer (9.10), Aditi Rao (8.75) and Priya Menon (8.05).

7.2 Views

Two views were created for the placement cell. **vw_department_placement_stats** summarizes, for each department, the number of students, applications and offers; **vw_company_offer_stats** summarizes, for each company, the number of drives, applications and offers. Views hide the multi-table joins behind a simple name, so a report is obtained with a one-line SELECT, and they provide a stable interface if the underlying tables later change.

8. Application Development

8.1 Architecture

Both applications follow a three-layer structure, shown in Figure 8.1. The presentation layer collects input and displays results, the service layer holds the application logic (eligibility display, transactions, error mapping), and the data-access layer opens JDBC connections to Oracle. Because the integrity rules live in the database, even a faulty front end cannot insert invalid data.

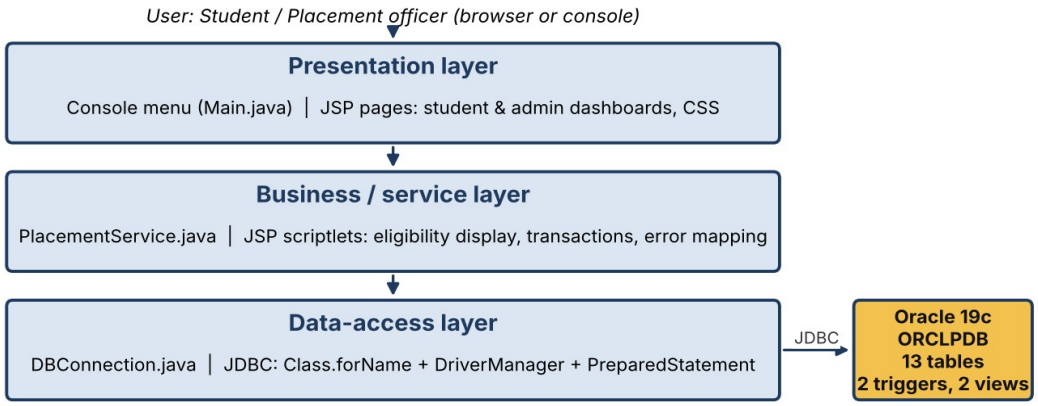


Figure 8.1: Layered architecture of the applications

8.2 Java Console Application

The console application is organised in three classes: DBConnection (opens the connection), PlacementService (all database operations) and Main (the text menu). It uses PreparedStatement for every parameterised query, which both avoids SQL injection and lets the database reuse execution plans. Operations that change several tables switch off auto-commit with setAutoCommit(false), call commit() on success and rollback() when an SQLException occurs. ORA error codes are translated into readable messages; for example, ORA-20001 becomes "student does not meet the eligibility criteria". Menu option 1 lists the eligible students of a drive; for drive 3 it listed Aditi Rao (8.75, 0), Sneha Iyer (9.10, 0) and Priya Menon (8.05, 0), and an attempt to enrol Karthik Nair was reported as rejected.

8.3 JSP Web Portal

The web portal is a JSP application deployed on Tomcat under the context path PlacementPortal. It contains a landing page with two role cards, a student area and an administrator area.

Table 8.1: Pages of the web portal

Page	Role	Function
index.jsp	All	Landing page; choose Student or Placement Cell
student/login.jsp	Student	Login by student ID
student/dashboard.jsp	Student	Profile summary, list of drives with eligibility criteria, Eligible / Not eligible badge, Apply button, own applications and results
student/apply.jsp	Student	Inserts an application inside a transaction and reports the outcome
admin/login.jsp	Admin	Login with demo credentials
admin/dashboard.jsp	Admin	All drives with statistics and links to manage each drive
admin/drive-detail.jsp	Admin	Applicants of a drive with round badges, result entry and offer buttons
admin/enter-result.jsp	Admin	Records a Pass/Fail result; the round trigger validates the order
admin/issue-offer.jsp	Admin	Creates an offer for an application

The student dashboard is where the eligibility criteria become visible. For each drive it prints the minimum CGPA and maximum backlogs and compares them with the logged-in student's record using the rule cgpa >= min_cgpa and backlogs <= max_backlogs. The Apply button is displayed only when the student is eligible and has not yet applied. Even if a user crafted a request by hand, the trigger would still reject an ineligible application, so the interface hint and the database rule agree.

The administrator pages record round results and issue offers. Drives, job profiles and eligibility rows are loaded into the database with SQL scripts in this version; the portal itself does not create them. New identifiers are generated with SELECT NVL(MAX(id),0)+1, which is adequate for a single-user demonstration; a production system would use Oracle sequences.

8.4 Deployment Steps

1. Copy ojdbc8.jar into WEB-INF/lib of the PlacementPortal application.
2. Copy the PlacementPortal folder into the webapps directory of Tomcat (C:\xampp\tomcat\webapps).
3. Make sure the Oracle listener is running and ORCLPDB is open.
4. Start Tomcat from the XAMPP control panel.

5. Open http://localhost:8080/PlacementPortal/ in a browser.

An HTTP 500 error "No suitable driver found" appeared at first. The cause and the fix are described in Section 9.

8.5 Security Notes

Parameterised statements are used throughout, so user input is never concatenated into SQL. The administrator login uses hard-coded demonstration credentials and the student login uses only the student ID; both are acceptable for a coursework prototype but would be replaced in production by hashed passwords stored in a table, role-based access and a dedicated low-privilege database account instead of SYSTEM. The database password is kept in the JSP files for convenience and should be moved to a configuration source in a real deployment.

9. Challenges and Resolutions

Table 9.1: Problems and resolutions

Problem	Cause	Resolution
ORA-00942 table or view does not exist	Session connected as SYS while tables were created in the SYSTEM schema	Reconnect as SYSTEM or prefix table names with SYSTEM
ORA-01109 database not open	Pluggable database ORCLPDB was closed	ALTER PLUGGABLE DATABASE ORCLPDB OPEN
ORA-01017 invalid username/password	The SYSTEM password had been changed for another project on the same Oracle instance	Log in with the current password; both projects share the instance without conflict because each uses its own tables
Trigger created with compilation errors (PLS-00225)	Table named Round clashed with the ROUND function	Renamed the table to Placement_Round and recreated the trigger
HTTP 500 "No suitable driver" in the portal	JDBC driver was not registered inside Tomcat	Added Class.forName("oracle.jdbc.OracleDriver") before each getConnection and ojdbc8.jar in WEB-INF/lib; set pageEncoding UTF-8
Portal still showed old behaviour after the fix	The previous zip file had been extracted again	Replace the folder with the corrected version and restart Tomcat

One general lesson is to read the real error text before guessing: an early assumption that a special dash character caused the trigger failure was wrong, and SHOW ERRORS pointed directly to the actual cause.

10. Conclusion and Future Work

10.1 Conclusion

The project delivers a complete placement-management back end in Oracle 19c: thirteen normalized tables, declarative constraints, two triggers that enforce eligibility and round order, analytical queries and reporting views. On top of it, a Java JDBC console application and a JSP web portal with student and administrator dashboards show how an application can rely on the database for correctness. Testing confirmed that an ineligible student is rejected at the database level and that results cannot skip rounds.

10.2 Future Work

- Secure authentication with hashed passwords and role-based access control.
- Oracle sequences or identity columns instead of MAX+1 identifiers.
- Screens for administrators to create drives, job profiles and eligibility criteria.
- E-mail or SMS notifications for new drives, shortlists and offers.
- Resume upload, skill-based matching of students to job profiles and graphical placement analytics.
- A connection pool (for example Tomcat JNDI DataSource) in place of one connection per request.

References

1. Oracle Corporation, Oracle Database SQL Language Reference, 19c.
2. Oracle Corporation, Oracle Database PL/SQL Language Reference, 19c.
3. Oracle Corporation, Oracle Database JDBC Developer's Guide, 19c.
4. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson.
5. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill.
6. Apache Software Foundation, Apache Tomcat Documentation and Jakarta Server Pages specification.

Appendix A: Complete DDL Script

The script below was run as SYSTEM in ORCLPDB. The selection-round table was originally created as Round and later renamed to Placement_Round (Section 5.7); the listing shows the final name.

```
CREATE TABLE Department (
    dept_id     NUMBER PRIMARY KEY,
    dept_name   VARCHAR2(50) NOT NULL,
    dept_code   VARCHAR2(10) UNIQUE
);
CREATE TABLE Student (
    student_id  NUMBER PRIMARY KEY,
    name        VARCHAR2(60) NOT NULL,
    dept_id     NUMBER NOT NULL,
    email       VARCHAR2(80) UNIQUE,
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
);
CREATE TABLE Academic_Record (
    record_id   NUMBER PRIMARY KEY,
    student_id  NUMBER NOT NULL UNIQUE,          -- 1:1 with Student
    cgpa        NUMBER(3,2) NOT NULL,
    backlogs    NUMBER(2) DEFAULT 0 NOT NULL,
    CONSTRAINT fk_record_student FOREIGN KEY (student_id)
        REFERENCES Student(student_id),
    CONSTRAINT chk_cgpa CHECK (cgpa BETWEEN 0 AND 10),
    CONSTRAINT chk_backlogs CHECK (backlogs >= 0)
);
CREATE TABLE Skill (
    skill_id    NUMBER PRIMARY KEY,
    skill_name  VARCHAR2(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE Student_Skill (
    student_id  NUMBER NOT NULL,
    skill_id    NUMBER NOT NULL,
    CONSTRAINT pk_student_skill PRIMARY KEY (student_id, skill_id),
    CONSTRAINT fk_ss_student FOREIGN KEY (student_id)
        REFERENCES Student(student_id),
    CONSTRAINT fk_ss_skill FOREIGN KEY (skill_id)
        REFERENCES Skill(skill_id)
);
CREATE TABLE Company (
    company_id  NUMBER PRIMARY KEY,
    name        VARCHAR2(80) NOT NULL,
    sector      VARCHAR2(50)
);
CREATE TABLE Job_Profile (
    profile_id  NUMBER PRIMARY KEY,
    company_id  NUMBER NOT NULL,
    role        VARCHAR2(60) NOT NULL,
    package_lpa NUMBER(5,2),
    CONSTRAINT fk_profile_company FOREIGN KEY (company_id)
        REFERENCES Company(company_id)
);
CREATE TABLE Drive (
    drive_id    NUMBER PRIMARY KEY,
    profile_id  NUMBER NOT NULL,
    drive_date  DATE NOT NULL,
    CONSTRAINT fk_drive_profile FOREIGN KEY (profile_id)
        REFERENCES Job_Profile(profile_id)
);
CREATE TABLE Eligibility_Criteria (
    drive_id        NUMBER PRIMARY KEY,
    min_cgpa        NUMBER(3,2) NOT NULL,
    max_backlogs    NUMBER(2) DEFAULT 0,
    CONSTRAINT fk_elig_drive FOREIGN KEY (drive_id)
        REFERENCES Drive(drive_id),
    CONSTRAINT chk_min_cgpa CHECK (min_cgpa BETWEEN 0 AND 10)
);
CREATE TABLE Application (
    app_id      NUMBER PRIMARY KEY,
    student_id  NUMBER NOT NULL,
    drive_id    NUMBER NOT NULL,
    applied_on  DATE DEFAULT SYSDATE,
    CONSTRAINT fk_app_student FOREIGN KEY (student_id)
        REFERENCES Student(student_id),
    CONSTRAINT fk_app_drive FOREIGN KEY (drive_id)
        REFERENCES Drive(drive_id),
    CONSTRAINT uq_student_drive UNIQUE (student_id, drive_id)   -- one application per
student per drive
);
CREATE TABLE Placement_Round (
    round_id    NUMBER PRIMARY KEY,
    drive_id    NUMBER NOT NULL,
    round_no    NUMBER(2) NOT NULL,
    round_type  VARCHAR2(30) NOT NULL,     -- e.g. 'Aptitude', 'GD', 'Technical', 'HR'
    CONSTRAINT fk_round_drive FOREIGN KEY (drive_id)
        REFERENCES Drive(drive_id),
    CONSTRAINT uq_drive_roundno UNIQUE (drive_id, round_no)
);
CREATE TABLE Result (
    app_id      NUMBER NOT NULL,
    round_id    NUMBER NOT NULL,
    status      VARCHAR2(10) NOT NULL,     -- 'Pass' / 'Fail'
    CONSTRAINT pk_result PRIMARY KEY (app_id, round_id),
    CONSTRAINT fk_result_app FOREIGN KEY (app_id)
```

```
        REFERENCES Application(app_id),
    CONSTRAINT fk_result_round FOREIGN KEY (round_id)
        REFERENCES Placement_Round(round_id),
    CONSTRAINT chk_status CHECK (status IN ('Pass', 'Fail'))
);
CREATE TABLE Offer (
    offer_id    NUMBER PRIMARY KEY,
    app_id      NUMBER NOT NULL UNIQUE,     -- at most one offer per application
    offer_date  DATE DEFAULT SYSDATE,
    status      VARCHAR2(10) DEFAULT 'Pending',
    CONSTRAINT fk_offer_app FOREIGN KEY (app_id)
        REFERENCES Application(app_id),
    CONSTRAINT chk_offer_status CHECK (status IN ('Pending', 'Accepted', 'Rejected'))
);
```

Listing A.1: Table definitions

Appendix B: Sample Data

Representative INSERT statements for the data described in Section 6.

```
INSERT INTO Department VALUES (1,'Computer Science','CSE');
INSERT INTO Department VALUES (2,'Electronics','ECE');
INSERT INTO Department VALUES (3,'Mechanical','MECH');
INSERT INTO Student VALUES (101,'Aditi Rao',1,'aditi.rao@example.com');
INSERT INTO Student VALUES (102,'Rahul Verma',1,'rahul.verma@example.com');
INSERT INTO Student VALUES (103,'Sneha Iyer',2,'sneha.iyer@example.com');
INSERT INTO Student VALUES (104,'Karthik Nair',2,'karthik.nair@example.com');
INSERT INTO Student VALUES (105,'Priya Menon',3,'priya.menon@example.com');
-- Academic_Record: (student, CGPA, backlogs)
101: 8.75, 0   102: 7.20, 1   103: 9.10, 0   104: 6.80, 2   105: 8.05, 0
-- Skills: Java, Python, SQL, Communication (7 Student_Skill rows)
-- Companies / profiles
TechNova Solutions (IT Services)       - Software Engineer, 8.50 LPA
GreenGrid Energy (Renewable Energy)    - Electrical Design Engineer, 6.75 LPA
FinEdge Analytics (Fintech)            - Data Analyst, 7.20 LPA
-- Drives and eligibility
Drive 1: 15-OCT-2026, min CGPA 7.00, max backlogs 1
Drive 2: 20-OCT-2026, min CGPA 6.50, max backlogs 2
Drive 3: 25-OCT-2026, min CGPA 8.00, max backlogs 0
-- Applications: 1 (101->D1)  2 (102->D1)  3 (103->D2)  4 (105->D3)
-- Rounds: D1 Aptitude/Technical/HR (1-3)  D2 Aptitude/Technical (4-5)  D3 Technical/HR (6-
7)
-- Results (9 rows): app 1 Pass x3; app 2 round 1 Pass, round 2 Fail;
--                   app 3 Pass x2; app 4 Pass x2
-- Offers: 1 (app 1, Accepted)  2 (app 3, Pending)  3 (app 4, Accepted)
```

Listing B.1: Sample data summary